

# Image Recognition to Identify Species of Flowers

**Nsisong Patrick Ekong**

**Understanding Artificial Intelligence**

August 25, 2024

## Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                         | <b>4</b>  |
| <b>2</b> | <b>Methods</b>                              | <b>4</b>  |
| 2.1      | Data Preprocessing . . . . .                | 4         |
| 2.2      | Model Architecture . . . . .                | 4         |
| 2.3      | Evaluation Metrics: . . . . .               | 5         |
| <b>3</b> | <b>Analysis and Results</b>                 | <b>5</b>  |
| 3.1      | Dataset . . . . .                           | 5         |
| 3.2      | Model 1: . . . . .                          | 5         |
| 3.2.1    | Architecture and Hyperparameters: . . . . . | 5         |
| 3.2.2    | Model Performance . . . . .                 | 5         |
| 3.3      | Model 2: . . . . .                          | 5         |
| 3.3.1    | Architecture and Hyperparameters: . . . . . | 7         |
| 3.3.2    | Model Performance . . . . .                 | 7         |
| 3.4      | Model 3: . . . . .                          | 9         |
| 3.4.1    | Architecture and Hyperparameters: . . . . . | 9         |
| 3.4.2    | Model Performance . . . . .                 | 10        |
| 3.5      | Model 4: . . . . .                          | 10        |
| 3.5.1    | Architecture and Hyperparameters: . . . . . | 10        |
| 3.5.2    | Model Performance . . . . .                 | 12        |
| <b>4</b> | <b>Discussion</b>                           | <b>13</b> |
| <b>5</b> | <b>Conclusion</b>                           | <b>14</b> |

**Abstract**

This report examines the use of Convolutional Neural Networks (CNNs) to identify flower species from images. By utilizing a dataset of labeled flower images, the study involves training a CNN model and evaluating its classification performance. The process includes image preprocessing, model architecture design, and performance evaluation through metrics like accuracy and precision. The findings indicate that the CNN model is effective in accurately identifying flower species, although considerations such as model robustness and potential overfitting are discussed. The report concludes with suggestions for future research, including more complex models and transfer learning to enhance accuracy.

# 1 Introduction

In recent years, advances in machine learning, particularly deep learning, have transformed computer vision, with Convolutional Neural Networks (CNNs) becoming a powerful tool for image recognition tasks like identifying flower species. This task is challenging due to the wide variety of species, color similarities, and varying image conditions (Hiary et al., 2018). CNNs excel in this domain by automatically learning hierarchical features from images, achieving high accuracy, especially when trained on large datasets (Archana and Jeevaraj, 2024). Transfer learning, where pre-trained models on large datasets like ImageNet are fine-tuned for specific tasks, further enhances performance, particularly with limited labeled data (Zhao et al, 2024). Other approaches, such as combining CNNs with Recurrent Neural Networks (RNNs) to capture temporal and spatial dependencies, and using data augmentation to improve model robustness, have also been explored (Li et al, 2023; Xie et al, 2022). While unsupervised learning methods have been considered, supervised deep learning models generally offer better accuracy and reliability. Despite progress, challenges like variability in flower appearance and the need for high-resolution images to distinguish similar species remain. Addressing these often requires combining techniques like multi-scale CNNs or integrating contextual information (Wu et al, 2023). This report details the implementation of a CNN model using TensorFlow to classify flower species, aiming to train a model that can accurately distinguish between species.

## 2 Methods

### 2.1 Data Preprocessing

To prepare the dataset for training, several preprocessing steps were applied:

- **Resizing:** Standardizes image dimensions to a uniform size, facilitating consistent input for CNNs and optimizing computational efficiency.
- **Normalization:** Scales pixel values to a range like  $[0, 1]$ , accelerating training and improving convergence by standardizing the input data.
- **Data Augmentation:** Creates variations of the original images through transformations like rotations and flips, enhancing the model’s ability to generalize and reducing overfitting.

### 2.2 Model Architecture

The CNN models were built using TensorFlow and Keras. The architecture consists of several layers designed to extract features from the images:

- **Convolutional Layers:** Extract features from images by applying filters, enabling the network to detect patterns like edges and textures.
- **Pooling Layers:** Reduce the spatial dimensions of feature maps to decrease computational load and retain essential features.
- **Fully Connected Layers:** Perform classification by integrating features from convolutional and pooling layers to make final predictions.
- **Regularization:** Prevents overfitting by randomly setting a fraction of input units to zero during training, promoting generalization.

## 2.3 Evaluation Metrics:

The following metrics are used:

- **Accuracy:** Measures the proportion of correctly classified samples out of the total.
- **Precision:** Ratio of true positives to total predicted positives.
- **Recall:** Ratio of true positives to total actual positives.
- **F1 Score:** Harmonic mean of precision and recall.
- **Confusion Matrix:** Shows true positives, false positives, true negatives, and false negatives, providing detailed performance insight.

## 3 Analysis and Results

### 3.1 Dataset

The dataset used for this project is the TensorFlow Flowers dataset, which contains images of flowers belonging to five different species: Dandelion, Daisy, Roses, Sunflowers, and Tulips. The dataset is pre-labeled and consists of approximately 3,670 images. 80% of the dataset was used for training, and 20% was reserved for validation to monitor the model's performance on unseen data. In this analysis, four models are fitted as follows;

### 3.2 Model 1:

As a foundational model, the following is considered.

#### 3.2.1 Architecture and Hyperparameters:

The CNN model starts with a Rescaling layer to normalize input images, followed by three convolutional blocks with Conv2D layers, ReLU activation, BatchNormalization, and MaxPooling. The filter counts increase from 32 to 128 across these blocks. A Dropout layer with a 0.3 rate is applied before flattening the output. A dense layer with 256 units, followed by another BatchNormalization and a 0.5 Dropout layer, leads to a final softmax output layer with 5 units for classification. The architecture is designed to capture increasingly complex features, with BatchNormalization and Dropout used for regularization to prevent overfitting. Key hyperparameters such as learning rate, dropout rates, and filter numbers were tuned to optimize performance.

#### 3.2.2 Model Performance

The model summary is presented in Table 1. Figure 1 provides the plots of training and validation accuracy and plot of training and validation loss, and Figure 2 presents the Confusion matrix. The classification report is presented in Table 2.

### 3.3 Model 2:

As a review of the first model, a second model considered as presented below.

| Layer (type)                                | Output Shape         | Param #    |
|---------------------------------------------|----------------------|------------|
| rescaling_11 (Rescaling)                    | (None, 180, 180, 3)  | 0          |
| conv2d_36 (Conv2D)                          | (None, 180, 180, 32) | 896        |
| batch_normalization_39 (BatchNormalization) | (None, 180, 180, 32) | 128        |
| max_pooling2d_28 (MaxPooling2D)             | (None, 90, 90, 32)   | 0          |
| conv2d_37 (Conv2D)                          | (None, 90, 90, 64)   | 18,496     |
| batch_normalization_40 (BatchNormalization) | (None, 90, 90, 64)   | 256        |
| max_pooling2d_29 (MaxPooling2D)             | (None, 45, 45, 64)   | 0          |
| conv2d_38 (Conv2D)                          | (None, 45, 45, 128)  | 73,856     |
| batch_normalization_41 (BatchNormalization) | (None, 45, 45, 128)  | 512        |
| max_pooling2d_30 (MaxPooling2D)             | (None, 22, 22, 128)  | 0          |
| dropout_19 (Dropout)                        | (None, 22, 22, 128)  | 0          |
| flatten_11 (Flatten)                        | (None, 61,952)       | 0          |
| dense_16 (Dense)                            | (None, 256)          | 15,859,968 |
| batch_normalization_42 (BatchNormalization) | (None, 256)          | 1,024      |
| dropout_20 (Dropout)                        | (None, 256)          | 0          |
| outputs (Dense)                             | (None, 5)            | 1,285      |

Table 1: CNN Model 1 Summary

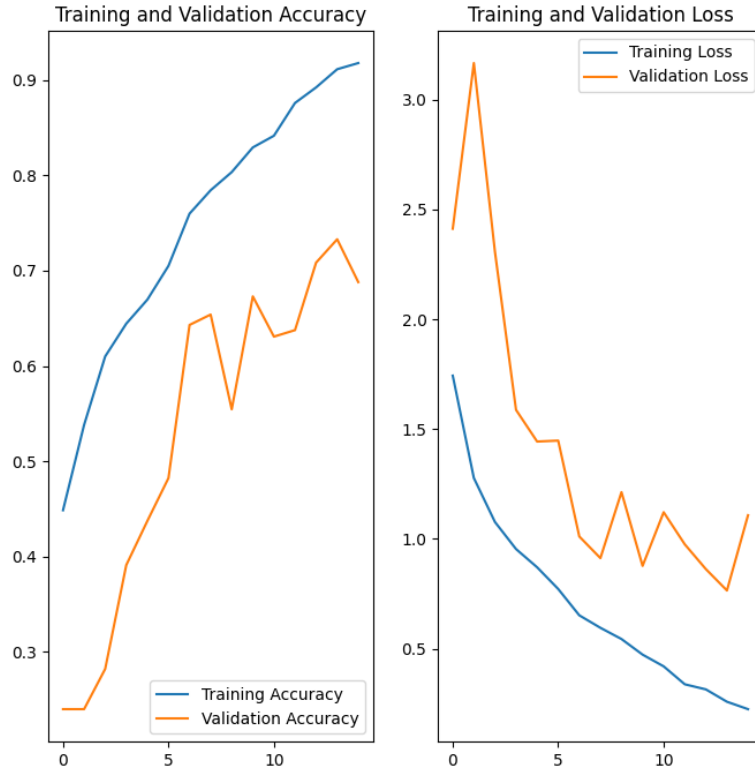


Figure 1: Plot Validation Accuracy &amp; Validation Loss against Epoch

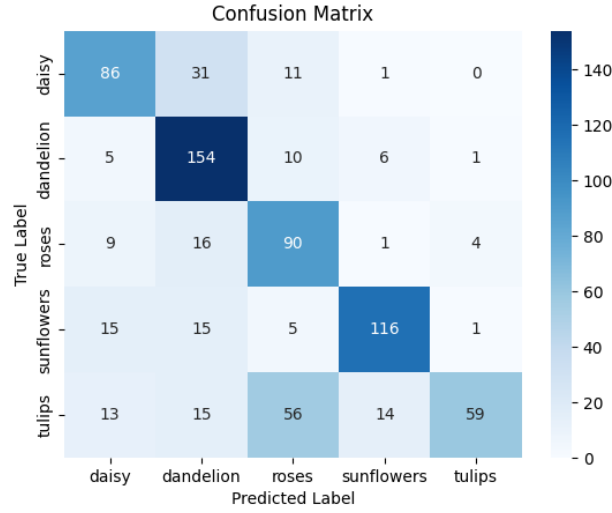


Figure 2: Model 1 Confusion Matrix

| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 0                   | 0.68      | 0.69   | 0.69     | 129     |
| 1                   | 0.75      | 0.77   | 0.76     | 176     |
| 2                   | 0.56      | 0.62   | 0.59     | 120     |
| 3                   | 0.83      | 0.83   | 0.83     | 152     |
| 4                   | 0.68      | 0.61   | 0.64     | 157     |
| <b>Accuracy</b>     |           |        | 0.71     | 734     |
| <b>Macro avg</b>    | 0.70      | 0.70   | 0.70     | 734     |
| <b>Weighted avg</b> | 0.71      | 0.71   | 0.71     | 734     |

Table 2: CNN Model 1 Classification report

### 3.3.1 Architecture and Hyperparameters:

The CNN model begins with a Rescaling layer to normalize input images, followed by three convolutional blocks with Conv2D layers, using ReLU activation, BatchNormalization, and MaxPooling. The filter sizes increase from 32 to 128 across these blocks, allowing for progressively complex feature extraction. A Dropout layer with a 0.3 rate is applied to reduce overfitting. The output is flattened and passed through a fully connected layer with 256 units, followed by BatchNormalization and a higher Dropout rate of 0.5. The final output layer uses softmax activation to classify the input into one of five classes. The architecture is designed to capture detailed features with increasing filter sizes, and ReLU is used for its training effectiveness. BatchNormalization and Dropout help stabilize training and prevent overfitting. The model's performance is optimized by adjusting key hyperparameters such as dropout rates, the number of filters, and the dense layer's units. The model was trained for 15 epochs, completing all without early stopping.

### 3.3.2 Model Performance

The model summary is presented in Table 3. Figure 3 provides the plots of training and validation accuracy and plot of training and validation loss, and Figure 4 presents the Confusion matrix. The

| Layer (type)                               | Output Shape        | Param # |
|--------------------------------------------|---------------------|---------|
| rescaling_2 (Rescaling)                    | (None, 180, 180, 3) | 0       |
| conv2d_3 (Conv2D)                          | (None, 90, 90, 32)  | 2,432   |
| batch_normalization_4 (BatchNormalization) | (None, 90, 90, 32)  | 128     |
| max_pooling2d_3 (MaxPooling2D)             | (None, 45, 45, 32)  | 0       |
| conv2d_4 (Conv2D)                          | (None, 23, 23, 64)  | 51,264  |
| batch_normalization_5 (BatchNormalization) | (None, 23, 23, 64)  | 256     |
| max_pooling2d_4 (MaxPooling2D)             | (None, 11, 11, 64)  | 0       |
| conv2d_5 (Conv2D)                          | (None, 6, 6, 128)   | 204,928 |
| batch_normalization_6 (BatchNormalization) | (None, 6, 6, 128)   | 512     |
| max_pooling2d_5 (MaxPooling2D)             | (None, 3, 3, 128)   | 0       |
| dropout_2 (Dropout)                        | (None, 3, 3, 128)   | 0       |
| flatten_1 (Flatten)                        | (None, 1152)        | 0       |
| dense_2 (Dense)                            | (None, 256)         | 295,168 |
| batch_normalization_7 (BatchNormalization) | (None, 256)         | 1,024   |
| dropout_3 (Dropout)                        | (None, 256)         | 0       |
| dense_3 (Dense)                            | (None, 5)           | 1,285   |

Table 3: CNN Model 2 Summary

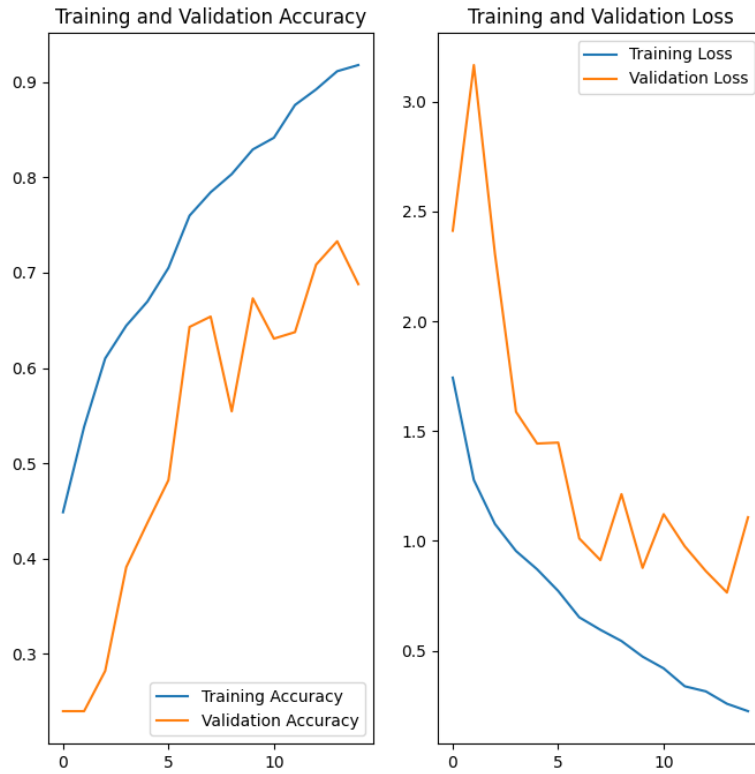


Figure 3: Plot Validation Accuracy &amp; Validation Loss against Epoch



classification report is presented in Table 4.

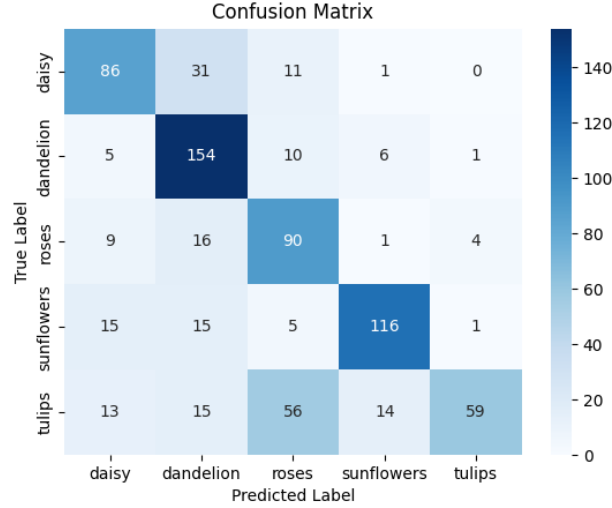


Figure 4: Model 2 Confusion Matrix

| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 0                   | 0.67      | 0.67   | 0.67     | 129     |
| 1                   | 0.67      | 0.88   | 0.76     | 176     |
| 2                   | 0.52      | 0.75   | 0.62     | 120     |
| 3                   | 0.84      | 0.76   | 0.80     | 152     |
| 4                   | 0.91      | 0.38   | 0.53     | 157     |
| <b>Accuracy</b>     |           |        | 0.69     | 734     |
| <b>Macro avg</b>    | 0.72      | 0.69   | 0.67     | 734     |
| <b>Weighted avg</b> | 0.73      | 0.69   | 0.68     | 734     |

Table 4: CNN Model 2 Classification Report

### 3.4 Model 3:

A review of the second model births the third as presented below.

#### 3.4.1 Architecture and Hyperparameters:

The CNN model is designed to capture increasingly complex features while managing overfitting. It begins by normalizing input images with a Rescaling layer and includes three convolutional blocks with filter sizes of 32, 64, and 128, using 5x5 kernels and 2x2 strides for efficient downsampling. Each convolutional layer is followed by BatchNormalization to stabilize training and MaxPooling to reduce spatial dimensions. To combat overfitting, a Dropout layer with a 0.4 rate is added after the convolutional blocks. The output is flattened and passed through a fully connected layer with 256 units, followed by another BatchNormalization and a higher Dropout rate of 0.5. The final layer uses softmax activation to classify the input into one of five categories. The model's architecture is optimized through fine-tuning key hyperparameters like kernel size, strides, and dropout rates, which are crucial for enhancing generalization and classification accuracy.

### 3.4.2 Model Performance

The model summary is presented in Table 5. Figure 5 provides plots of training and validation accuracy and plot of training and validation loss, and Figure 6 presents the Confusion matrix. The classification report is presented in Table 6.

| Layer (type)                                | Output Shape        | Param # |
|---------------------------------------------|---------------------|---------|
| rescaling_15 (Rescaling)                    | (None, 180, 180, 3) | 0       |
| conv2d_48 (Conv2D)                          | (None, 90, 90, 32)  | 2,432   |
| batch_normalization_55 (BatchNormalization) | (None, 90, 90, 32)  | 128     |
| max_pooling2d_37 (MaxPooling2D)             | (None, 45, 45, 32)  | 0       |
| conv2d_49 (Conv2D)                          | (None, 23, 23, 64)  | 51,264  |
| batch_normalization_56 (BatchNormalization) | (None, 23, 23, 64)  | 256     |
| max_pooling2d_38 (MaxPooling2D)             | (None, 11, 11, 64)  | 0       |
| conv2d_50 (Conv2D)                          | (None, 6, 6, 128)   | 204,928 |
| batch_normalization_57 (BatchNormalization) | (None, 6, 6, 128)   | 512     |
| max_pooling2d_39 (MaxPooling2D)             | (None, 3, 3, 128)   | 0       |
| dropout_27 (Dropout)                        | (None, 3, 3, 128)   | 0       |
| flatten_15 (Flatten)                        | (None, 1152)        | 0       |
| dense_21 (Dense)                            | (None, 256)         | 295,168 |
| batch_normalization_58 (BatchNormalization) | (None, 256)         | 1,024   |
| dropout_28 (Dropout)                        | (None, 256)         | 0       |
| outputs (Dense)                             | (None, 5)           | 1,285   |

Table 5: CNN Model 3 Summary

| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 0                   | 0.86      | 0.50   | 0.63     | 129     |
| 1                   | 0.67      | 0.69   | 0.68     | 176     |
| 2                   | 0.66      | 0.38   | 0.48     | 120     |
| 3                   | 0.85      | 0.54   | 0.66     | 152     |
| 4                   | 0.44      | 0.87   | 0.58     | 157     |
| <b>Accuracy</b>     |           |        | 0.61     | 734     |
| <b>Macro avg</b>    | 0.70      | 0.60   | 0.61     | 734     |
| <b>Weighted avg</b> | 0.69      | 0.61   | 0.61     | 734     |

Table 6: CNN Model 3 Classification Report

## 3.5 Model 4:

Tuning the hyperparameters of Model 3 ushers in the third as presented below.

### 3.5.1 Architecture and Hyperparameters:

The CNN model starts with a rescaling layer to normalize input images and consists of five convolutional blocks, each with 3x3 convolutions and increasing filters from 32 to 512. Each convolutional

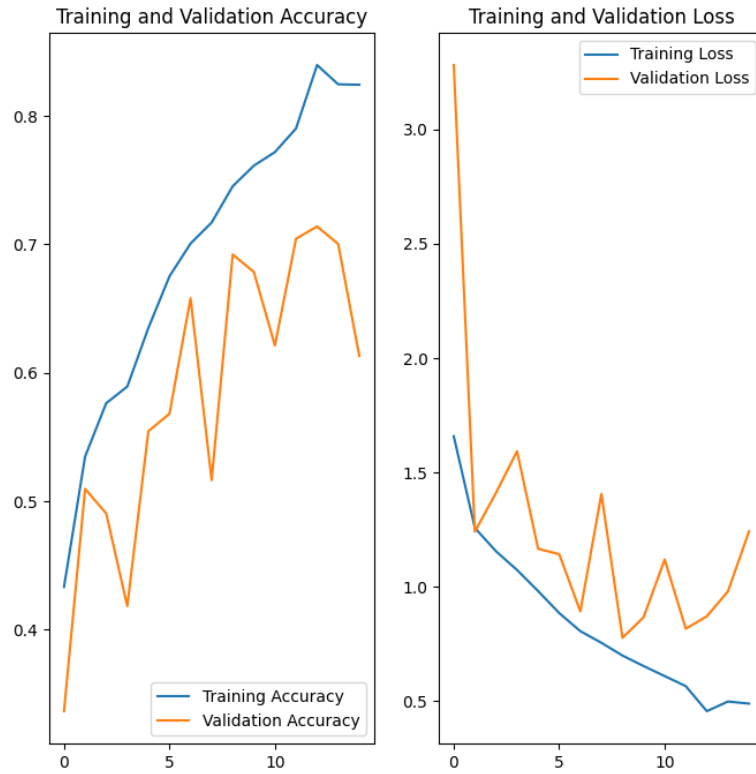


Figure 5: Plot Validation Accuracy &amp; Validation Loss against Epoch

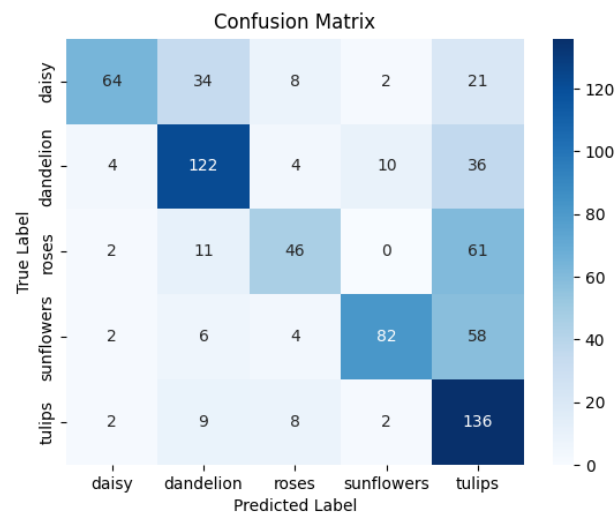


Figure 6: Model 3 Confusion Matrix

layer uses ReLU activation, followed by batch normalization and max pooling to reduce spatial dimensions and stabilize training. To prevent overfitting, dropout layers are included, and after flattening,

the output passes through a dense layer with 512 units before the final softmax layer, which classifies images into five categories. The model addresses class imbalance using SMOTE to generate synthetic samples and applies class weights to emphasize minority classes during training. Hyperparameter tuning involves adjusting the learning rate with an Adam optimizer, determining the batch size, and setting dropout rates to optimize performance. A learning rate scheduler is used to manage changes in the learning rate over epochs, improving convergence, while the batch size and dropout rates are fine-tuned to balance learning and generalization.

### 3.5.2 Model Performance

The model summary is presented in Table 7. Figure 7 provides plots of training and validation accuracy and plot of training and validation loss, and Figure 8 presents the Confusion matrix. The classification report is presented in Table 8.

| Layer (type)                                | Output Shape         | Param #   |
|---------------------------------------------|----------------------|-----------|
| rescaling_13 (Rescaling)                    | (None, 180, 180, 3)  | 0         |
| conv2d_37 (Conv2D)                          | (None, 180, 180, 32) | 896       |
| batch_normalization_48 (BatchNormalization) | (None, 180, 180, 32) | 128       |
| max_pooling2d_37 (MaxPooling2D)             | (None, 90, 90, 32)   | 0         |
| conv2d_38 (Conv2D)                          | (None, 90, 90, 64)   | 18,496    |
| batch_normalization_49 (BatchNormalization) | (None, 90, 90, 64)   | 256       |
| max_pooling2d_38 (MaxPooling2D)             | (None, 45, 45, 64)   | 0         |
| conv2d_39 (Conv2D)                          | (None, 45, 45, 128)  | 73,856    |
| batch_normalization_50 (BatchNormalization) | (None, 45, 45, 128)  | 512       |
| max_pooling2d_39 (MaxPooling2D)             | (None, 22, 22, 128)  | 0         |
| conv2d_40 (Conv2D)                          | (None, 22, 22, 256)  | 295,168   |
| batch_normalization_51 (BatchNormalization) | (None, 22, 22, 256)  | 1,024     |
| max_pooling2d_40 (MaxPooling2D)             | (None, 11, 11, 256)  | 0         |
| conv2d_41 (Conv2D)                          | (None, 11, 11, 512)  | 1,180,160 |
| batch_normalization_52 (BatchNormalization) | (None, 11, 11, 512)  | 2,048     |
| max_pooling2d_41 (MaxPooling2D)             | (None, 5, 5, 512)    | 0         |
| dropout_22 (Dropout)                        | (None, 5, 5, 512)    | 0         |
| flatten_11 (Flatten)                        | (None, 12800)        | 0         |
| dense_20 (Dense)                            | (None, 512)          | 6,554,112 |
| batch_normalization_53 (BatchNormalization) | (None, 512)          | 2,048     |
| dropout_23 (Dropout)                        | (None, 512)          | 0         |
| outputs (Dense)                             | (None, 5)            | 2,565     |

Table 7: CNN Model 4 Summary

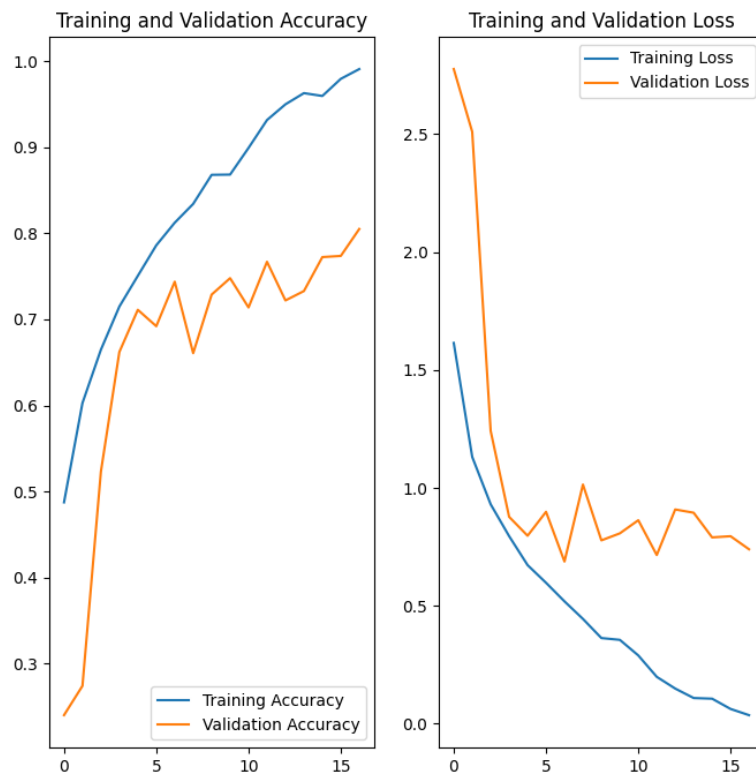


Figure 7: Plot Validation Accuracy &amp; Validation Loss against Epoch

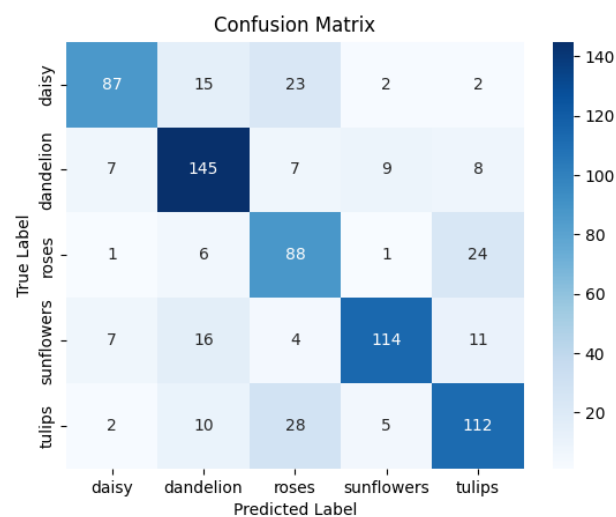


Figure 8: Model 4 Confusion Matrix

## 4 Discussion

The CNN models underwent several iterations to enhance performance and generalization. Model 1 used a basic architecture with three convolutional layers and standard regularization techniques like

| Class               | Precision | Recall | F1-Score | Support |
|---------------------|-----------|--------|----------|---------|
| 0                   | 0.81      | 0.79   | 0.80     | 129     |
| 1                   | 0.84      | 0.82   | 0.83     | 176     |
| 2                   | 0.69      | 0.68   | 0.69     | 120     |
| 3                   | 0.79      | 0.93   | 0.86     | 152     |
| 4                   | 0.82      | 0.73   | 0.77     | 157     |
| <b>Accuracy</b>     |           |        | 0.80     | 734     |
| <b>Macro avg</b>    | 0.79      | 0.79   | 0.79     | 734     |
| <b>Weighted avg</b> | 0.80      | 0.80   | 0.79     | 734     |

Table 8: CNN Model 4 Classification Report

Dropout and BatchNormalization. It achieved 71% accuracy but struggled with overfitting due to the lack of early stopping. Model 2 improved feature extraction by increasing the kernel size to 5x5 and the stride to 2x2, and raised the dropout rate to 0.4. Despite these changes, the model’s accuracy decreased slightly to 69%, and overfitting persisted without early stopping. Model 3 incorporated data augmentation, a learning rate scheduler, and early stopping. However, the overall accuracy fell to 61%, with the model showing high precision but lower recall, suggesting it was still overfitting and not generalizing well as evident in Figure 5. Model 4 introduced SMOTE and class weights to address class imbalance, deepened the architecture to five convolutional layers, and continued using early stopping and a learning rate scheduler. This model achieved the highest accuracy of 80% and exhibited balanced performance across all classes, indicating effective mitigation of overfitting and improved generalization.

## 5 Conclusion

This report details the application of Convolutional Neural Networks (CNNs) for classifying flower species from images. Four CNN models were developed and evaluated using the TensorFlow Flowers dataset. The study demonstrates that CNNs are effective for this image classification task, achieving notable accuracy and precision across different models. Overall, CNNs prove to be a powerful tool for flower species classification, with promising results and room for future enhancements.

## References

- [1] Archana, R. & Jeevaraj, P.S.E., (2024). Deep learning models for digital image processing: a review. *Artificial Intelligence Review*, 57(11). Available at: <https://doi.org/10.1007/s10462-023-10631-z> [Accessed 09 Aug. 2024].
- [2] Hiary, H., Saadeh, H., Saadeh, M., & Yaqub, M., (2018). Flower classification using deep convolutional neural networks. *IET Computer Vision*, 12(6), pp. 855-862.
- [3] Li, M., Jiang, Y., Zhang, Y. & Zhu, H., (2023). Medical image analysis using deep learning algorithms. *Frontiers in Public Health*, 11, p.1273253. Available at: <https://doi.org/10.3389/fpubh.2023.1273253> [Accessed 12 Aug. 2024].
- [4] Wu, H., Fang, L., Yu, Q., Yuan, J. & Yang, C., (2023). Plant leaf identification based on shape and convolutional features. *Expert Systems with Applications*, 219, p.119626. Available at: <https://doi.org/10.1016/j.eswa.2023.119626>.
- [5] Xie, F., Yuan, H., Ning, Y., Ong, M.E.H., Feng, M., Hsu, W., Chakraborty, B. & Liu, N., (2022). Deep learning for temporal data representation in electronic health records: A systematic review of challenges and methodologies. *Journal of Biomedical Informatics*, 126, p.103980. Available at: <https://doi.org/10.1016/j.jbi.2021.103980> [Accessed 10 Aug. 2024].
- [6] Zhao, Z., Alzubaidi, L., Zhang, J., Duan, Y. & Gu, Y., (2024). A comparison review of transfer learning and self-supervised learning: Definitions, applications, advantages and limitations. *Expert Systems with Applications*, 242, p.122807. Available at: <https://doi.org/10.1016/j.eswa.2023.122807> [Accessed 10 Aug. 2024].